

Universidade do Minho

Escola de Engenharia

Otimização de Inferência de LLMs em Sistemas Fujitsu A64FX

Análise de Dados de Elevado Desempenho Trabalho Prático II

Grupo 12

Humberto Gil Azevedo Sampaio Gomes PG59806

José António Fernandes Alves Lopes PG59807

José António Costa Soares PG60277

14 de maio de 2026

Resumo

Com o aumento do uso de ambientes de supercomputação para cargas de IA, a escassez de GPUs no supercomputador Deucalion, obriga, por vezes, à execução de tarefas de inferência em CPUs. Procurou-se medir e otimizar o desempenho da inferência de LLMs num único nó ARM do Deucalion, equipado com um processador Fujitsu A64FX.

Através da *engine llama.cpp*, foram avaliados modelos como o *SmolLM-2-135M*, *Gemma-3-4B* e *Meta-Llama-3.1-8B* em diferentes quantizações (Q3_K.M, Q4_K.M e Q8_0), compiladores (GCC, Clang, FCC) e configurações de *threading*. Os resultados demonstram que a combinação Clang + BLIS reduz o *Time Per Output Token* (TPOT) em até 68.5%, enquanto que o ajuste da política NUMA (*distribute*) melhora o desempenho em 4,4%. No entanto, a quantização não apresentou padrões consistentes de ganho, e problemas de replicabilidade e escala na avaliação da qualidade das respostas limitaram a análise. Apesar dos ganhos obtidos, há ainda potencial para melhorias adicionais no ajuste de diversos outros parâmetros das LLMs (tamanho do *batch*, μ -*batch*, contexto, etc.).

Além da otimização empírica, foi desenvolvido um modelo analítico de desempenho baseado na largura de banda da memória. No entanto, este subestima os tempos observados por fatores de $7\times$ a $19\times$ devido a limitações não só na sua formulação como também na sua adequação à arquitetura ARMv8.

1 Introdução

Nos últimos anos, as cargas de trabalho de inteligência artificial (IA) tornaram-se cada vez mais predominantes em supercomputadores, cujo uso já não se limita a aplicações tradicionais de computação científica. Entre 2021 e 2025, as cargas de treino e inferência de IA representaram cerca de 11% das horas de computação atribuídas na rede EuroHPC [1]. O supercomputador Deucalion [2], inserido nesta rede, dispõe de um número limitado de GPUs (geralmente mais indicadas para estas cargas), que não é suficiente para suprir a procura, obrigando os seus utilizadores a executar cargas de IA em sistemas baseados em CPUs.

Neste contexto, este trabalho visa otimizar processos de inferência em CPU, utilizando grandes modelos de linguagem (LLMs) como caso de estudo. Pretende-se executar diferentes modelos num nó ARM do Deucalion, equipado com um processador Fujitsu A64FX [3], utilizando o *llama.cpp* [4] como *engine* de inferência. Serão exploradas e afinadas variáveis como o compilador utilizado, o nível de quantização dos modelos e configurações de *threading*. Além disso, pretende-se construir um modelo que permita prever o desempenho da inferência nestes ambientes.

2 Background

A inferência de LLMs é um processo composto por duas fases distintas: *prefill* e *decode*. Na fase de *prefill*, os *tokens* da *prompt* são processados em paralelo, por meio de operações matriciais densas, resultando numa elevada intensidade aritmética e num processo *compute-bound* (limitado pela capaci-

dade de processamento aritmético). Já a fase de *decode*, que consiste na geração dos *tokens* da resposta, envolve operações matriz-vetor esparsas, com baixa reutilização de dados e natureza *memory-bound* (limitadas pela largura de banda da memória RAM) [5, 6].

Neste contexto, a quantização emerge como uma estratégia de otimização fundamental. Ao reduzir a precisão dos pesos para formatos numéricos com menor número de bits, o espaço ocupado pelos pesos diminui, o que reduz a quantidade de dados a transferir da memória e melhora o desempenho da geração de *tokens*. Contudo, essa técnica pode comprometer a qualidade das respostas geradas pelas LLMs.

Por outro lado, a eficiência computacional durante o *decoding* depende também da KV Cache. O mecanismo de atenção dos *transformers* requer as representações de *key* e *value* de todos os *tokens* precedentes [7]. Para resolver esse problema, mantém-se uma KV Cache, que armazena os tensores já computados. No entanto, a KV cache exige uma grande quantidade de memória, um recurso limitado, especialmente em cenários de nó único, como o analisado neste estudo. Mecanismos como **PagedAttention** visam atenuar este problema através da alocação dinâmica e reutilização de partes da KV cache [9].

As *engines* de inferência também exploram o paralelismo oferecido pelo *hardware*. No caso de inferência em CPUs, as *engines* utilizam instruções vetoriais e múltiplas *threads* em execução paralela. Além disso, técnicas como *Multi-token Prediction* (MTP) permitem a geração de vários *tokens* da mesma sequência em paralelo [8].

Por último, embora este estudo não analise o desempenho de pedidos concorrentes, esse é o cenário mais comum em ambientes de produção. Por isso, várias técnicas relevantes procuram extrair paralelismo para esse caso de uso. Um exemplo é o *Continuous Batching*, que permite ao servidor processar múltiplos pedidos simultaneamente. Dessa forma, é possível computar vários *tokens* de sequências diferentes na mesma iteração pelos pesos do modelo, amortizando o custo de leitura e aumentando significativamente o *throughput*.

3 Metodologia

Para avaliar o desempenho de inferência de LLMs de um modo geral, é necessário considerar o desempenho de vários modelos. Selecionaram-se os modelos apresentados na Tabela 1:

Modelo	Quantizações	Fonte
SmolLM-2-135M-Instruct	Q3_K_M, Q4_K_M, Q8_0	[11]
Gemma-3-4B-Instruct	Q3_K_M, Q4_K_M, Q8_0	[12]
Meta-Llama-3.1-8B-Instruct	Q3_K_M, Q4_K_M, Q8_0	[13]

Tabela 1: LLMs selecionadas para *benchmarking*.

Foram vários os critérios utilizados para a seleção destes modelos:

- Os modelos têm um número reduzido de parâmetros, capazes de caber na memória de um único nó ARM (32 GiB) juntamente com as KV caches;
- Os modelos são *open weight*, disponíveis não só para os autores, mas também para qualquer pessoa que pretenda replicar os nossos resultados;
- Os modelos são de tamanhos e arquiteturas distintos, possibilitando uma análise abrangente do desempenho;
- Os modelos encontram-se disponíveis em quantizações distintas, permitindo uma análise dos efeitos da quantização no desempenho e na qualidade das respostas;
- Os modelos são do tipo **Instruct**, adequados para inferência orientada por instruções [14].

A *engine* de inferência selecionada para a análise de desempenho foi o `llama.cpp` [4], devido à sua facilidade de utilização e ao seu excelente suporte para inferência em CPU. Esta escolha também exige que os modelos da Tabela 1 se encontrem disponíveis no formato GGUF, o que se verifica.

No âmbito deste trabalho, foram conduzidas diversas experiências com o objetivo de avaliar diferentes aspetos das LLMs, desde o seu desempenho até à qualidade das suas respostas. Um aspeto comum a todos os testes foi o ambiente de execução: todos os testes foram executados em nós da partição **normal-arm** do Deucalion, sendo cada modelo executado num único nó. Cada nó está equipado com um processador Fujitsu A64FX, com 48 núcleos ARMv8.2-A a 2.0 GHz, e ainda de 32 GiB de memória de alta largura de banda (HBM2) [2, 3]. Como a largura de banda da memória é geralmente o fator limitante para o desempenho de inferência [5], este foi o principal motivo para a utilização dos nós ARM em detrimento dos nós x86 também presentes no Deucalion. As versões do *software* relevante utilizado encontram-se apresentadas na Tabela 2.

<i>Software (Runtime)</i>	Versão	<i>Software (Compilação)</i>	Versão
Sistema Operativo	Rocky Linux 8.5	GNU C Compiler (GCC)	13.3.0
Kernel Linux	4.18.0-348.23.1.el8_5.aarch64	Clang	19.1.7
<code>llama.cpp</code>	b8849	Fujitsu C Compiler (FCC)	4.8.0
BLIS	1.0	CMake	3.31.3

Tabela 2: Versões do *software* relevante utilizado.

Para a execução de qualquer teste, cria-se um servidor **llama-server**, submetem-se as *prompts* através de pedidos HTTP à sua *Completions API* [15], e recolhem-se as respostas do servidor para extrair o *output* das LLMs e métricas de desempenho. As configurações do **llama-server** utilizadas para os vários modelos são apresentadas na Tabela 3. Estas correspondem às configurações predefinidas de cada modelo, com duas exceções:

- O número máximo de *tokens* a gerar foi definido como 4096, para evitar ciclos e respostas intermináveis;
- A *flag* `--no-mmap` foi utilizada para forçar o carregamento total dos modelos para memória, o que reduziu a frequência de *crashes* observados para modelos maiores, fenómeno que será discutido posteriormente.

	SmolLM2	Gemma 3	Llama 3.1
Temperatura	0.8		
Top-K	40		
Top-P	0.95		
Tipos de dados da KV cache	f16		
Máximo de <i>tokens</i> gerados	4096		
Tamanho do Contexto	8192	131072	
Tamanho do Batch	2048		
Tamanho do μ -batch	512		

Tabela 3: Configuração do `llama.cpp` utilizada para cada modelo.

3.1 Testes de Desempenho

Estes testes têm como objetivo avaliar o desempenho dos modelos para uma dada configuração do `llama.cpp`, independentemente da qualidade das respostas geradas. Para isso, cada nó ARM executa uma instância de um `llama-server` com um único modelo e quantização. São enviadas, em sequência, várias *prompts* para o servidor (uma de cada vez), e são recolhidas métricas sobre o desempenho do modelo para a execução de cada *prompt*. Para garantir a replicabilidade dos resultados, a *seed* de aleatoriedade é fixada em zero.

As *prompts* utilizadas são as S0, M0, e L0 do Anexo A. Elas são enviadas ao servidor nesta ordem, cada uma repetida dez vezes, e o servidor é reiniciado entre diferentes tipos de *prompt*, para garantir que não ocorre reuso da KV cache entre estes. Estas dez execuções são necessárias para que, posteriormente, seja possível selecionar quais *runs* devem ser descartadas para *warmup* e quais devem ser utilizadas para representar o desempenho do modelo. Para cada *prompt* executada, as seguintes métricas de desempenho são extraídas da resposta do `llama-server` [15]:

- Número de *tokens* da *prompt* processados;
- Número de *tokens* da *prompt* reutilizados da KV cache;
- *Time To First Token* (TTFT) – tempo até a geração do primeiro *token*, estimado como a soma do tempo de processamento da *prompt* com o tempo médio de geração de um *token* (TPOT);
- *Time Per Output Token* (TPOT) – tempo médio de geração de um *token*;

Adicionalmente, são ainda recolhidas as seguintes métricas no final da execução de cada *prompt*:

- Uso de memória, através do comando `free` [16].

Resta determinar quantas *runs* são necessárias para *warmup* e para medições de desempenho. Para isso, realizaram-se testes de desempenho para todas as combinações possíveis de compilador, modelo, e quantização, e foi efetuada uma análise estatística para definir estes valores. Os valores de TPOT medidos revelaram-se bastante estáveis, sem diferenças significativas entre as primeiras execuções e as seguintes. No entanto, o mesmo já não se verificou para o TTFT, como ilustra a Figura 1.

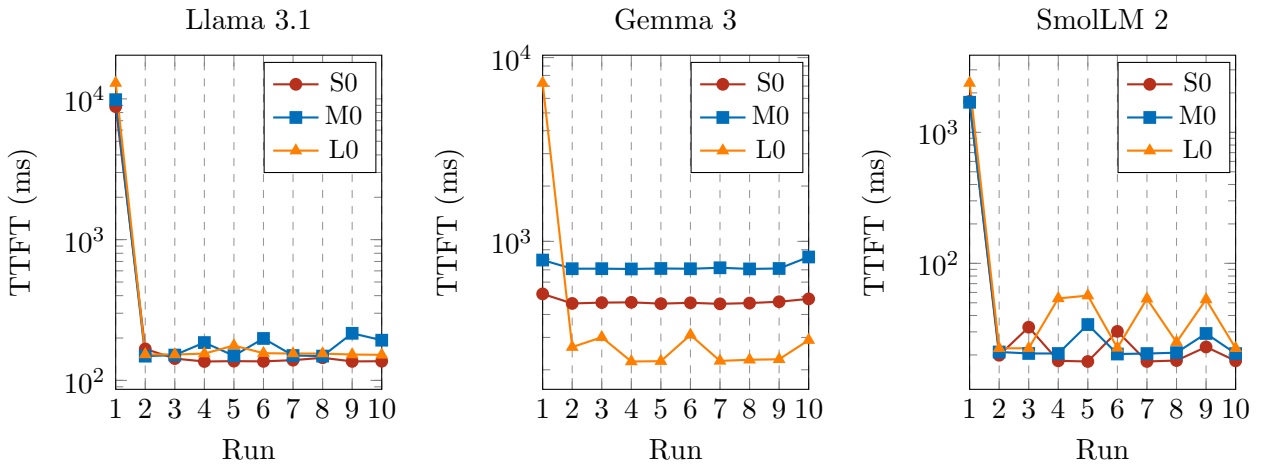


Figura 1: Valores de TTFT dos modelos testados em dez runs sucessivas das mesmas *prompts*, utilizando Clang + BLIS, quantização Q4_K_M, e 48 *threads*.

Como se pode observar, nos resultados do Llama 3.1 e do SmolLM 2, o primeiro valor de TTFT medido é várias ordens de grandeza superior aos restantes (repare-se na escala logarítmica dos gráficos). Este fenómeno, também observado para os restantes compiladores e quantizações, deve-se à utilização da KV Cache: na primeira execução, o número de *tokens* da *prompt* processado corresponde ao total de *tokens* da mesma; no entanto, nas execuções subsequentes, este valor reduz-se para 1, uma vez que os *tokens* restantes da *prompt* são reutilizados da KV Cache. Já no caso do Gemma 3, este comportamento não se verifica de forma tão acentuada: apenas se observam reduções significativas no número de *tokens* processados e, conseqüentemente, no TTFT, para a *prompt* L0, independentemente do compilador ou da quantização utilizados.

Foram realizadas experiências adicionais para estudar este fenómeno (como variar a *seed* e a *prompt*), e concluiu-se que a sua mitigação só é possível através da variação das *prompts* enviadas em sequência ao llama-server. Optou-se por não se alterar este aspeto na metodologia de *benchmarking*, de forma a capturar o melhor desempenho possível dos modelos, ainda que os resultados não reflitam cenários realistas, onde existe uma maior diversidade de *prompts*.

Assim, concluiu-se que é necessária uma execução inicial para *warmup*. Para a medição de desempenho, optou-se por recolher as cinco medições subsequentes, de forma a calcular a média e o desvio padrão

das diversas métricas. Este número mais elevado de medições é especialmente relevante para os valores de TTFT do SmoLLM 2, que, por serem reduzidos, estão sujeitos a maior interferência do sistema e, consequentemente, a uma maior variabilidade.

3.2 Testes de Qualidade de Resposta

Estes testes têm como objetivo avaliar a qualidade das respostas dos modelos, independentemente do seu desempenho. Para a sua execução, são utilizados servidores `llama-server`, configurados de acordo com a Tabela 3. As *prompts* no Anexo A são enviadas a todos os servidores, e é recolhida a resposta gerada por cada modelo para cada uma delas.

As *prompts* abrangem diversas áreas de conhecimento (matemática, informática, física, química, artes, *etc.*), permitindo uma avaliação abrangente dos modelos. Além disso, todas as *prompts* foram redigidas em Inglês, a única língua suportada por todas as LLMs testadas.

Para garantir a replicabilidade dos resultados, a *seed* de aleatoriedade é fixada em zero. Em teoria, isto assegura que cada modelo devolve sempre a mesma resposta para a mesma *prompt*, embora, como se verificará adiante, isso possa não ocorrer na prática.

Após a execução das LLMs, as respostas geradas são avaliadas por cada um dos autores deste trabalho, de acordo com a seguinte escala:

- 0 – A resposta é completamente incorreta e/ou não responde à *prompt*;
- 1 – A resposta responde corretamente à *prompt*, mas contém erros em informações auxiliares, como exemplos incorretos;
- 2 – A resposta responde corretamente à *prompt* sem qualquer erro ou alucinação.

Esta escala não permite distinguir a qualidade de diferentes respostas corretas. Por exemplo, duas respostas podem estar corretas, mas uma pode fornecer exemplos muito mais ilustrativos do que a outra. No entanto, devido ao tamanho reduzido dos modelos, o foco principal foi avaliar a correção das respostas, e não a sua qualidade, um aspeto mais relevante para modelos maiores.

Reconhecemos que esta metodologia apresenta várias limitações. Em primeiro lugar, os avaliadores, especialistas em informática, não são especialistas em todos os tópicos abordados, o que pode resultar em avaliações menos rigorosas para algumas *prompts*. Além disso, o número limitado de *prompts* e a falta de amostragem de *seeds* resultam numa visão restrita do modelo, uma vez que são analisadas apenas quinze respostas por modelo, um número insuficiente para uma avaliação abrangente. No entanto, devido a restrições de tempo e recursos humanos, não foi possível abordar estes problemas de forma mais aprofundada.

4 Otimizações Realizadas

Com uma metodologia de *benchmarking* definida, passou-se à definição de uma metodologia de otimização. Optou-se por uma abordagem em cascata, na qual se varia uma variável de cada vez, selecionando a configuração com melhor desempenho antes de se avançar para o ajuste da variável seguinte. Deste modo, reduz-se consideravelmente o número de medições necessárias, uma vez que não é preciso testar todas as combinações possíveis de todas as variáveis. No entanto, existe o risco de convergência para um ótimo local, ou seja, não se garante que o melhor desempenho global seja atingido. As variáveis foram ajustadas na seguinte ordem:

- Compilador utilizado para compilar o `llama.cpp`;
- Uso da *flag* `--no-mmap`;
- Quantização dos modelos;
- Configuração de *threads* e acesso a memória não-uniforme (NUMA).

4.1 Compilador Utilizado

O compilador utilizado para compilar um programa pode ter um impacto significativo no seu desempenho. Por isso, procurou-se comparar o desempenho do `llama.cpp` quando compilado com diferentes compiladores. Foram comparados os dois principais compiladores de C++ *open-source*: GCC [17] e Clang [18]. O `llama.cpp` também suporta o uso de bibliotecas BLAS para acelerar operações matriciais durante o processamento das *prompts* [19]. Por isso, também se testou a compilação do `llama.cpp` com as seguintes bibliotecas:

- OpenBLAS [20];
- BLIS [21];
- FJBLAS [22], uma biblioteca BLAS da Fujitsu otimizada para processadores A64FX, que apenas é suportada pelo compilador FCC da Fujitsu [23].

A compilação do `llama.cpp` é, geralmente, simples. No Deucalion, após carregar os módulos de compilação necessários, basta executar os seguintes comandos num nó ARM, na diretoria do código-fonte do `llama.cpp`:

```
$ export CC=gcc CXX=g++ # Definir compiladores
$ cmake -B build # Configurar compilacao
$ cmake --build build --config Release -j48 # Compilar llama.cpp
```


Por omissão, as *flags* de compilação são automaticamente otimizadas para o sistema. Neste caso, observou-se o uso da *flag* `-march=a64fx+sve`, o que indica que o código gerado foi otimizado para processadores A64FX com suporte para SVE.

Para adicionar suporte para bibliotecas BLAS, deve substituir-se a segunda linha do excerto acima por uma das seguintes, conforme a biblioteca BLAS a utilizar [19, 24]:

```
$ cmake -B build -DGGML_BLAS=ON -DGGML_BLAS_VENDOR=OpenBLAS          # OpenBLAS
$ cmake -B build -DGGML_BLAS=ON -DGGML_BLAS_VENDOR=FLAME              # BLIS
$ cmake -B build -DGGML_BLAS=ON -DGGML_BLAS_VENDOR=Fujitsu_SSL2BLAMP  # FJBLAS
```

Para certas configurações, a compilação decorreu sem qualquer problema. No entanto, como mostra a Tabela 4, não foi possível compilar o `llama.cpp` com a maioria das configurações desejadas. Em primeiro lugar, não foi possível *linkar* o OpenBLAS, uma vez que, na versão compilada nos módulos do Deucalion, apenas a interface de 64-bits (ILP64) é suportada, enquanto que o `llama.cpp` utiliza a interface tradicional de 32-bits. Além disso, também não foi possível utilizar o FCC: o modo tradicional (FCC-Trad) tem suporte limitado para *standards* recentes de C++, o que resultou em erros de *templating* durante a compilação do `llama.cpp`; já o modo Clang (FCC-Clang) falha durante a geração de código ao tentar produzir uma instrução máquina inválida.

BLAS / Compilador	GCC 13.3.0	Clang 19.1.7	FCC-Trad 4.8.0	FCC-Clang 4.8.0
Nenhuma	✓	✓	✗	✗
BLIS	✓	✓	✗	✗
OpenBLAS	✗	✗	✗	✗
FJBLAS	—	—	✗	✗

Tabela 4: Matriz de configurações de compilação do `llama.cpp` bem e mal sucedidas.

Procurou-se ainda utilizar o FJBLAS com outros compiladores, carregando a biblioteca BLAS da Fujitsu em *runtime* através da variável de ambiente `LD_PRELOAD`, utilizando uma versão do `llama.cpp` corretamente compilada. No entanto, esta abordagem não foi possível, uma vez que o FJBLAS está disponível apenas como uma biblioteca estática. Desta forma, o número de configurações de compilação testadas foi significativamente inferior ao pretendido.

Para avaliar os diferentes compiladores, foram realizados testes de desempenho (Secção 3.1), cujos resultados são apresentados na Figura 2. Verifica-se que, independentemente da utilização do BLIS, o Clang geralmente atinge valores de TPOT inferiores aos do GCC. Relativamente ao TTFT, observam-se semelhanças entre os resultados das diversas configurações de compilação. No entanto, as medições obtidas com o GCC, tanto para o TTFT como para o TPOT, apresentam maior instabilidade (com desvios padrão significativamente mais elevados), o que resulta em medições esporádicas mais elevadas, como é o caso do TTFT para *prompts* de dimensão média do Gemma 3. Por outro lado, não se verificaram diferenças significativas no uso de memória entre as diferentes configurações de compilador.

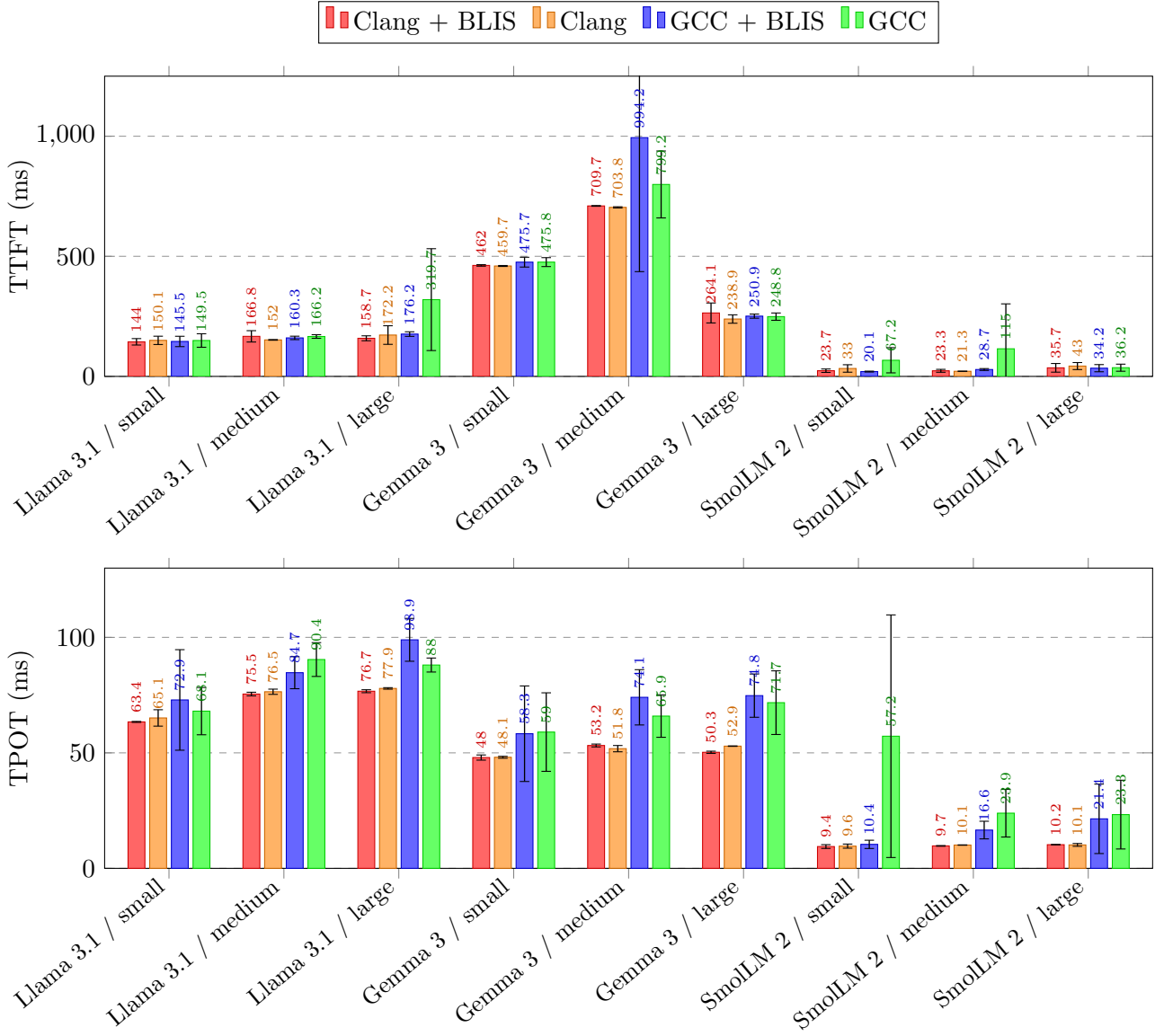


Figura 2: Comparação de diferentes compiladores no TTFT e TPOT de vários modelos, utilizando quantização Q4_KM e 48 threads.

Para a metodologia de otimização em cascata, é necessário selecionar um compilador para ser utilizado nas medições das etapas seguintes. Com o objetivo de efetuar esta escolha de forma estatisticamente fundamentada, cada medição foi, em primeiro lugar, normalizada em relação ao melhor tempo obtido para a sua configuração específica de modelo e *prompt*. De seguida, calculou-se a média geométrica por configuração de compilação, cujos resultados são apresentados na Tabela 5.

Configuração	TTFT	TPOT
Clang + BLIS	1.057	1.004
Clang	1.099	1.019
GCC + BLIS	1.104	1.371
GCC	1.555	1.691

Tabela 5: Média geométrica do TTFT e do TPOT, normalizados em relação ao melhor compilador por *benchmark*.

Observa-se que a configuração Clang + BLIS atinge os melhores tempos médios, tanto para o TTFT como para o TPOT, pelo que foi selecionada como base para as otimizações subsequentes.

Os testes anteriores revelaram que a diferença de desempenho entre as configurações com e sem BLIS era mínima. No entanto, esses resultados não são representativos de cenários realistas, uma vez que, como já foi explicado, grande parte do TTFT nas medições realizadas se deve à consulta de KV cache. Assim, procurou-se comparar o desempenho do BLIS considerando apenas a execução de *warmup* de cada modelo, onde todos os *tokens* da *prompt* são efetivamente processados. Os resultados, apresentados na Figura 3, mostram que o impacto do BLIS para o desempenho é frequentemente negativo, especialmente no Llama 3.1 e na SmolLM 2, aumentando o TTFT, em contraste com o que é indicado na documentação do `llama.cpp` [19].

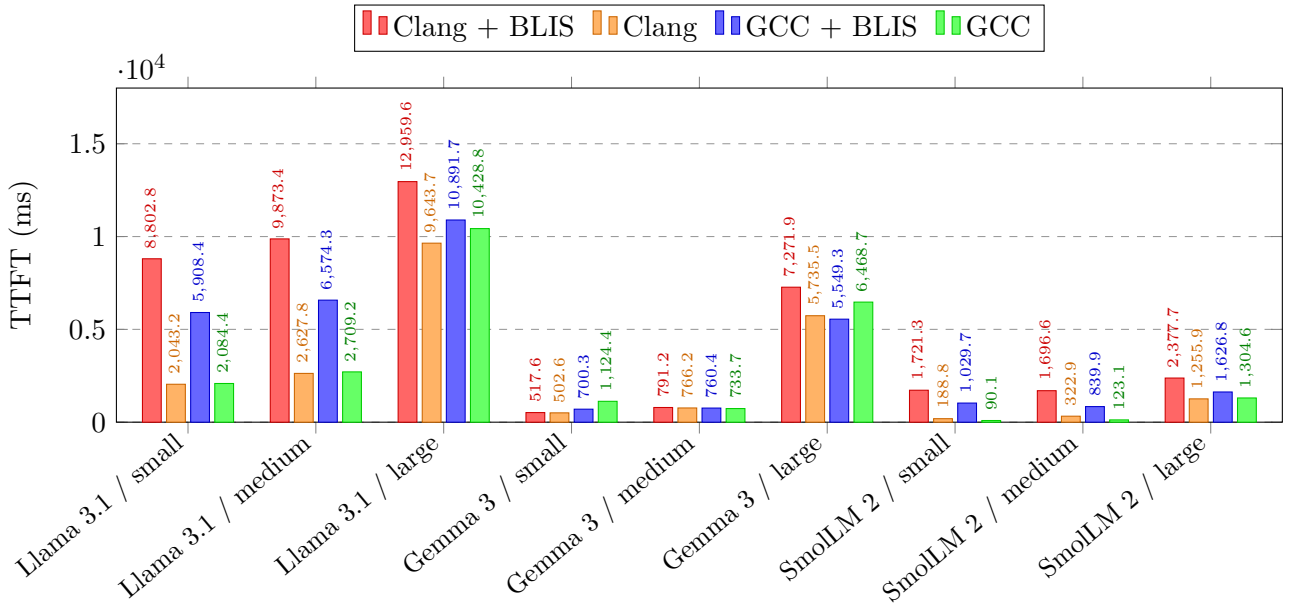
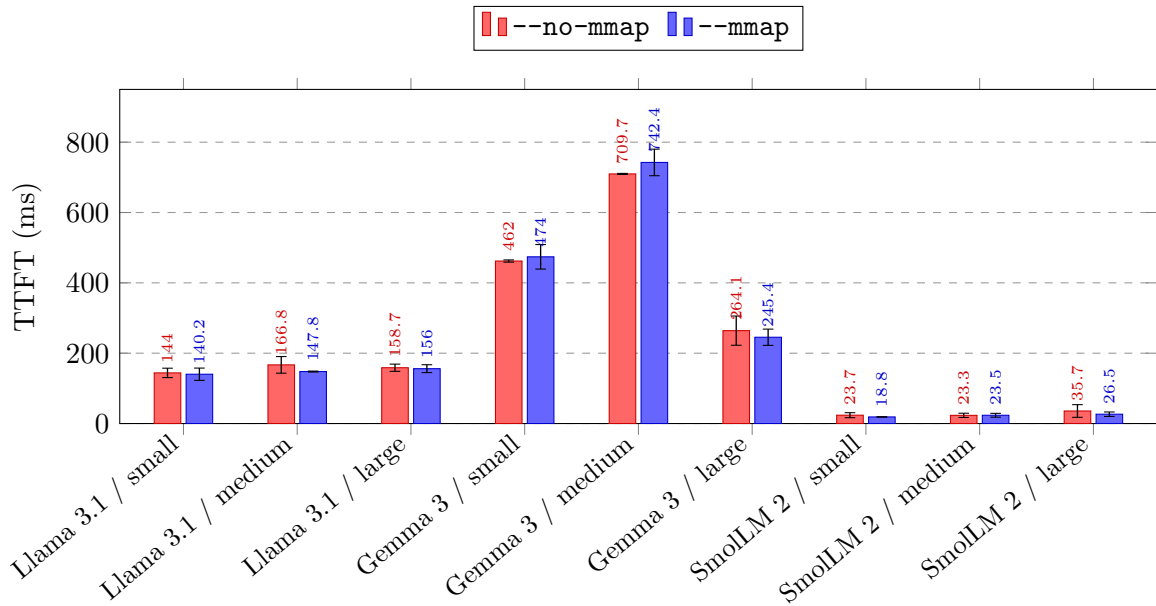


Figura 3: Comparação de diferentes compiladores no TTFT de *warmup* de vários modelos, utilizando quantização Q4_K_M e 48 *threads*.

4.2 Mapeamento do Modelo em Memória

Anteriormente, foi referido que modelos de maior dimensão, em particular o Llama 3.1 (em qualquer quantização), tinham tendência a causar um *crash* do `llama-server` com um `BUS ERROR` durante a sua inicialização. A nossa hipótese é que este fenómeno esteja relacionado com a duplicação de páginas entre o *kernel* e o *userspace*, o que conduz a um maior consumo de memória e, conseqüentemente, ao término abrupto do servidor. No entanto, não foi possível validar esta hipótese, uma vez que o *crash* ocorria de forma demasiado rápida para permitir a captura de amostras suficientes do uso de memória. O uso da *flag* `--no-mmap` reduziu significativamente a frequência destes *crashes*, embora não tenha eliminado o problema por completo.

Ainda assim, é importante demonstrar, através de testes de desempenho (Secção 3.1), que o uso do `--no-mmap` não compromete o desempenho das LLMs. A Figura 4 confirma isso: embora o consumo de memória com `--no-mmap` seja superior (devido à necessidade de carregar os pesos dos modelos para memória), não se verificam diferenças significativas nos valores de TTFT e TPOT entre as configurações `--mmap` e `--no-mmap`. Através de vários testes *t* de Student, não foi possível detetar uma diferença estatisticamente significativa nos valores médios do TTFT e TPOT dos diversos modelos.



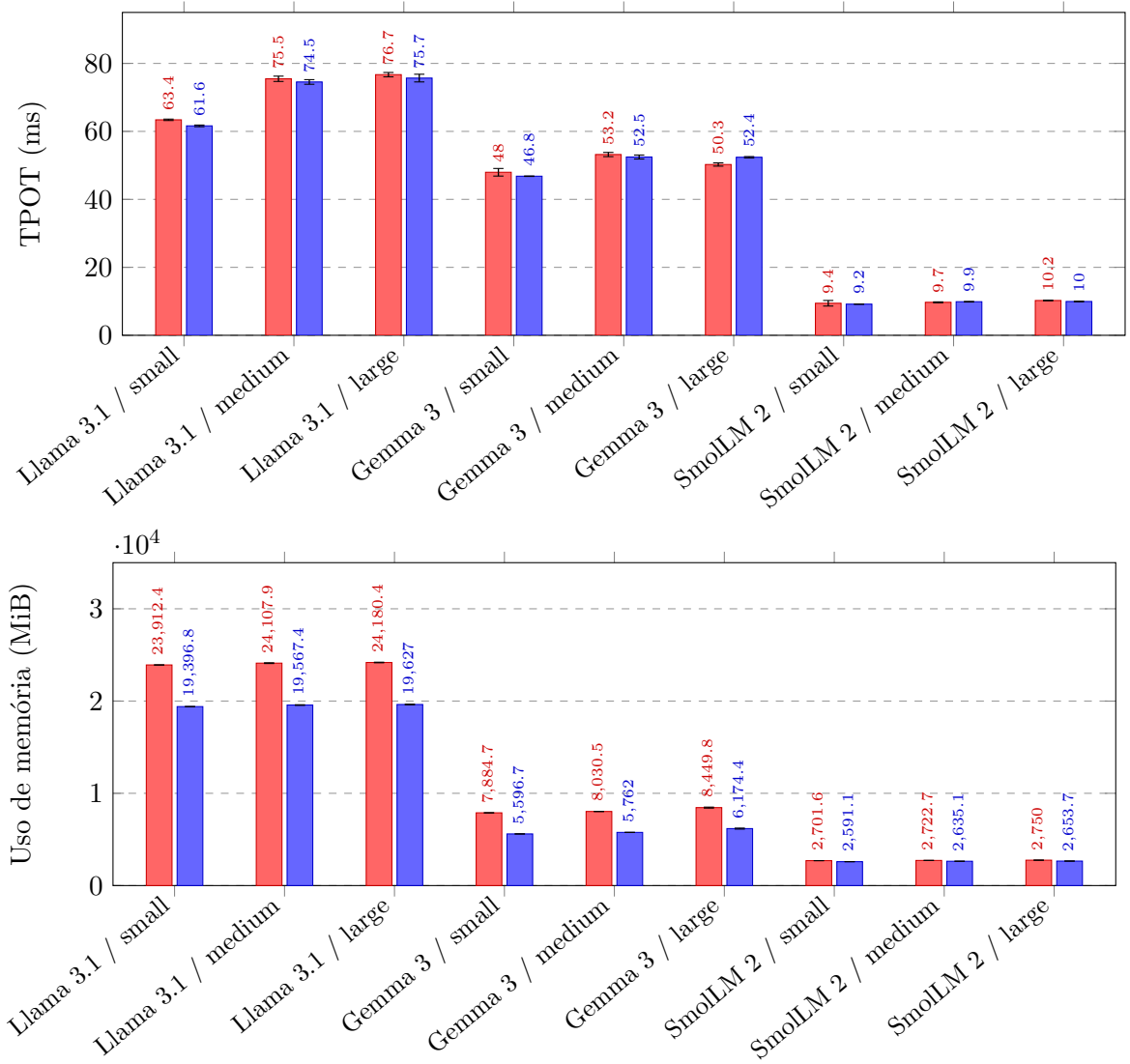


Figura 4: Comparação do TTFT, TPOT e uso de memória do `llama.cpp` com as flags `--mmap` e `--no-mmap`, utilizando Clang + BLIS, quantização Q4_K_M e 48 threads.

Assim, manteve-se o uso da flag `--no-mmap` para as otimizações seguintes.

4.3 Quantização

De seguida, procurou-se avaliar se a quantização dos modelos poderia melhorar o desempenho dos mesmos, bem como as eventuais consequências para a qualidade das respostas geradas. Para tal, começou-se pela avaliação da qualidade das respostas dos modelos quantizados (Secção 3.2), cujos resultados sumários são apresentados na Tabela 6.

		Pequenas (/ 30)	Médias (/ 30)	Grandes (/ 30)	Total (/ 90)
SmolLM 2	Q3_K_M	12	2	5	19
	Q4_K_M	6	3	6	15
	Q8_0	12	4	3	19
Gemma 3	Q3_K_M	27	24	26	77
	Q4_K_M	27	28	30	85
	Q8_0	26	26	27	79
Llama 3.1	Q3_K_M	30	23	29	82
	Q4_K_M	23	27	26	76
	Q8_0	29	23	29	81

Tabela 6: Pontuações sumárias das várias quantizações dos modelos num teste de análise de qualidade de respostas. As pontuações discriminadas por *prompt* e por avaliador podem ser encontradas no Anexo B.

Os resultados obtidos não permitiram extrair conclusões que auxiliem na seleção de uma quantização para os vários modelos, uma vez que não foi identificado qualquer padrão consistente na qualidade das respostas. Contrário às expectativas dos autores, não se verificou um aumento na pontuação com o aumento do número de bits por parâmetro.

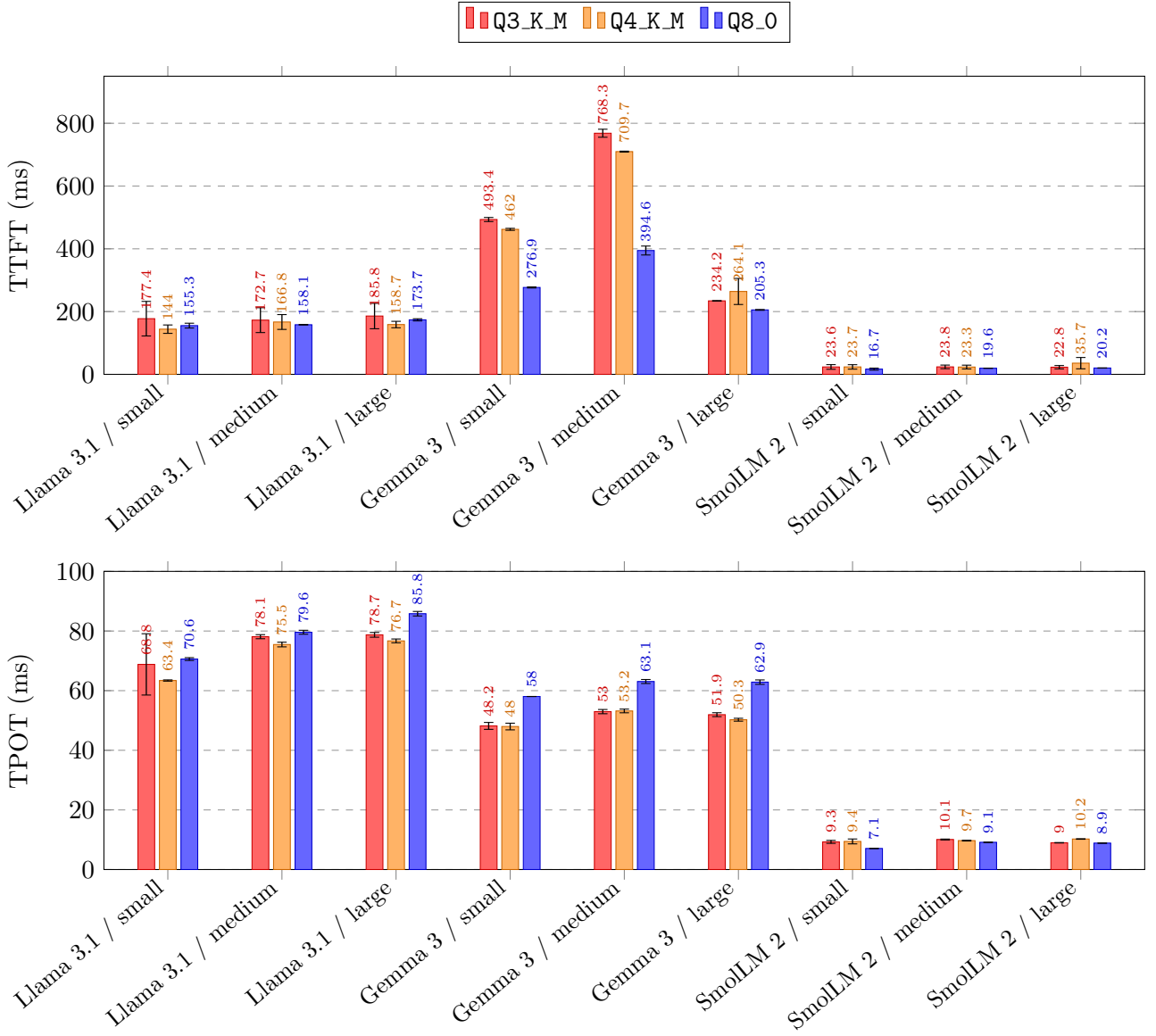
Como já foi referido, o número reduzido de *prompts*, *seeds* e avaliadores torna esta avaliação limitada, e suscetível a grande variabilidade nas pontuações devido a pequenas diferenças nas respostas, inerentes à natureza estocástica dos modelos. Por exemplo, o Llama 3.1 quantizado como Q4_K_M respondeu, para a *prompt* S4, que o segundo planeta mais próximo do Sol era Mercúrio, o que resultou na perda de seis pontos na sua avaliação. Este resultado seria menos relevante caso fossem testadas várias *seeds* de aleatoriedade, uma vez que o modelo não erra consistentemente.

A única conclusão relevante foi que o Gemma 3 e o Llama 3.1, apesar de apresentarem um número de parâmetros muito distinto, exibem uma qualidade de respostas semelhante. Este resultado demonstra que o número de pesos de um modelo não é o único fator que determina a sua qualidade, e que a sua arquitetura e protocolo de treino também a influenciam significativamente.

Outro problema identificado neste protocolo de testes foi a falta de replicabilidade. Embora todas as *prompts* enviadas ao `llama-server` sejam independentes, e cada resposta gerada com uma *seed* de aleatoriedade fixa, não se verificou que a resposta de cada modelo a cada *prompt* fosse sempre a mesma. Este fenómeno manifestou-se para as *prompts* de média e grande dimensão, em que a ordem de envio das *prompts* influenciava as respostas obtidas. Os autores não tiveram oportunidade de analisar o código-fonte do `llama.cpp` para investigar este bug em profundidade, mas pretendem fazê-lo e reportá-lo futuramente.

Deste modo, uma vez que os testes de qualidade das respostas não são replicáveis e não permitem uma comparação justa entre as diferentes quantizações, os seus resultados foram descartados. Assim,

o desempenho das várias quantizações será o único critério considerado para a seleção da quantização a utilizar nas fases seguintes do processo de otimização em cascata. Os resultados dos testes de desempenho realizados (Secção 3.1) encontram-se na Figura 5.



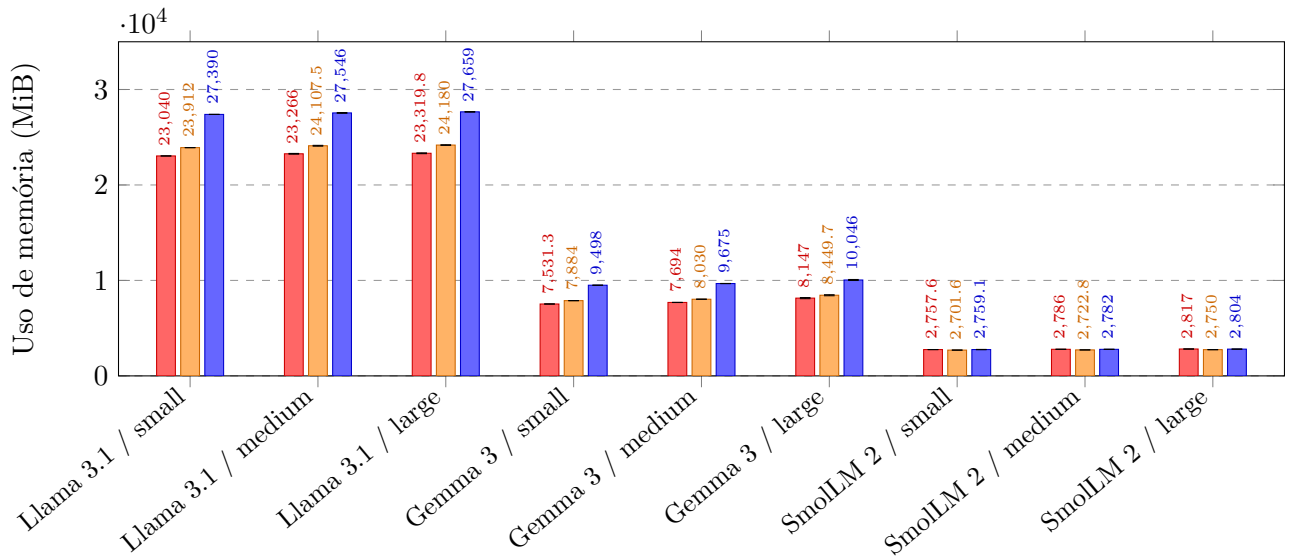


Figura 5: Comparação do TTFT, TPOT e uso de memória do `llama.cpp` entre diferentes quantizações dos modelos, utilizando Clang + BLIS e 48 *threads*.

A análise do TTFT teve como principal foco o model Gemma 3, para *prompts* de pequena e média dimensão, os únicos casos em que todos os *tokens* da *prompt* foram efetivamente todos processados nas medições recolhidas. Observa-se que o TTFT diminui à medida que o número de bits por parâmetro aumenta. Como o processamento da *prompt*, que domina o TTFT, é uma operação *compute-bound*, considera-se que o custo da conversão dos pesos quantizados para formatos numéricos suportados pelo processador seja a causa desta tendência. No entanto, validar esta hipótese exigiria um *profiling* detalhado dos *kernels* do `llama.cpp`, o que está fora do âmbito deste trabalho. No que diz respeito aos outros modelos, o TTFT é dominado por acessos à KV-Cache, o que justifica a semelhança dos valores de TTFT entre diferentes quantizações do mesmo modelo.

Em relação ao TPOT, o único padrão consistente observado foi que, para o modelo Gemma 3, a quantização Q8.0 apresenta um tempo médio por geração de *token* superior aos das restantes. Inesperadamente, não se verificou uma diferença significativa no desempenho entre as várias quantizações de outros modelos. A inferência de LLMs é um processo *memory-bound*, e esperava-se que, como a quantização altera consideravelmente o tamanho do segmento de memória (pesos do modelo) que deve ser acedido cada *token* gerado, houvesse uma maior diferença no TPOT de diferentes quantizações dos mesmos modelos. Esta discrepância será analisada mais adiante, na discussão do modelo de desempenho utilizado.

A análise do uso de memória é direta: quanto maior o número de bits por parâmetro, maior será o tamanho do modelo e maior o uso de memória. No entanto, a diferença no uso de memória nunca é muito significativa, uma vez que a maior parte da memória é alocada para a KV Cache, e não para os pesos do modelo.

Sem forma de corretamente avaliar nem desempenho dos modelos nem a qualidade das suas respostas, optou-se por se manter a quantização Q4_K.M para os testes seguintes.

4.4 Threading

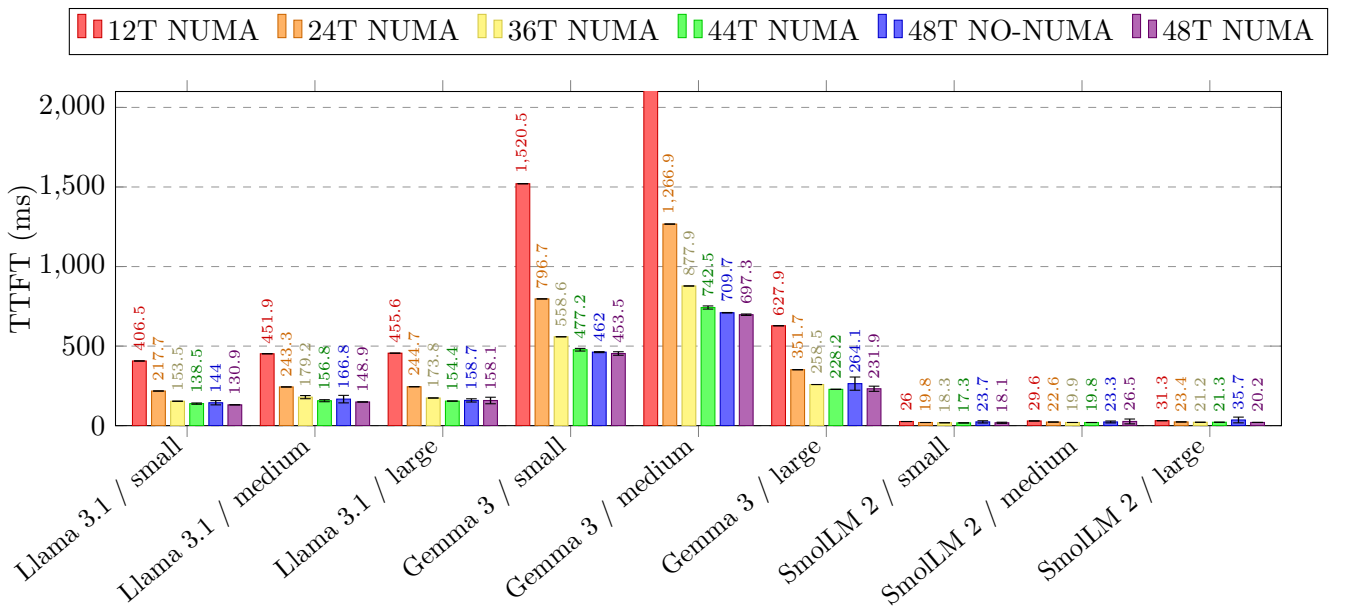
Após os ajustes à quantização dos modelos, procedeu-se ao ajuste do número de *threads* utilizados pelo `llama.cpp`. As configurações testadas foram 12, 24, 36 e 48 *threads*, correspondendo, respetivamente, a um, dois, três, e quatro quartos do número total de *cores* disponíveis. Além disso, testou-se também uma configuração de 44 *threads*, com o objetivo de manter alguns núcleos livres para operações do sistema operativo, reduzindo assim a sua interferência no desempenho do `llama.cpp`.

O sistema ARM utilizado possui memória de acesso não uniforme (NUMA) [3], ou seja, o tempo de acesso à memória varia consoante a localização de cada núcleo do processador em relação aos bancos de memória. Por este motivo, também se procurou estudar a influência das políticas de alocação NUMA no desempenho dos modelos. O `llama.cpp` suporta duas políticas de alocação para sistemas NUMA:

- **distribute**, em que os pesos do modelo e a KV cache são distribuídos pelos vários nós NUMA;
- **isolate**, em que o `llama.cpp` tenta alocar memória dentro de um único nó NUMA sempre que possível.

Em testes preliminares, a política **isolate** revelou-se várias ordens de grandeza mais lenta do que a **distribute**, tanto em TTFT como em TPOT. Isto faz sentido, tendo em conta que utilizar apenas um banco de memória do processador (quando possível, em modelos como o SmolLM 2), reduz a largura de banda da mesma em quatro. Logo, os resultados da política **isolate** não serão aqui apresentados.

Para as várias configurações de *threading* selecionadas, foram realizados testes de desempenho (Secção 3.1), cujos resultados obtidos são apresentados na Figura 6.



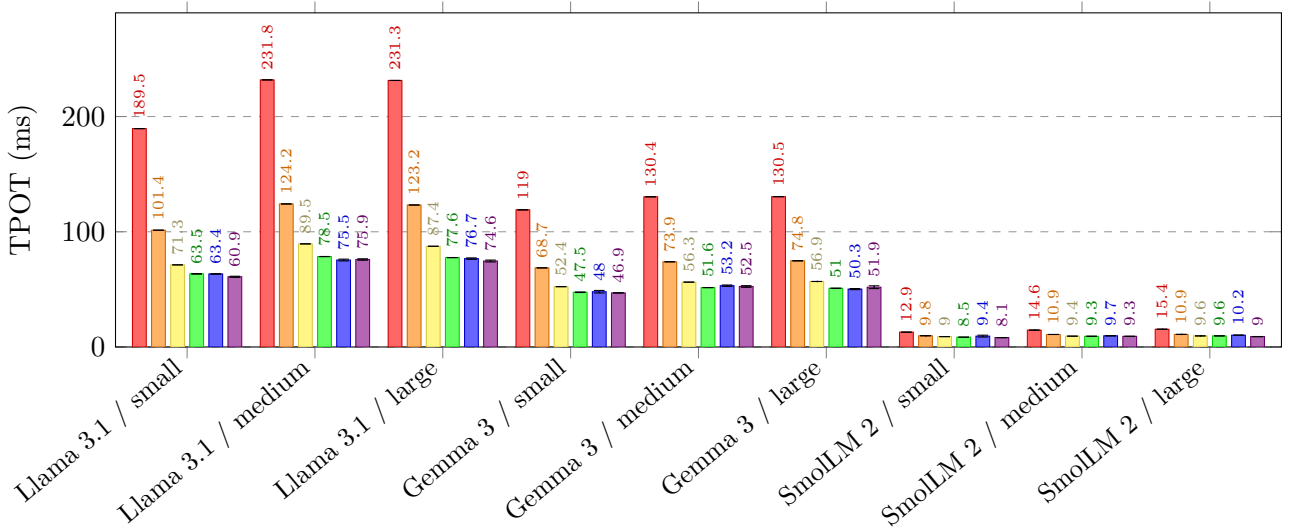


Figura 6: Comparação do TTFT e TPOT do `llama.cpp` entre configurações de *threading*, utilizando Clang + BLIS e Q4_K_M. A configuração *48T NO-NUMA* refere-se à configuração utilizada em todas as medições até agora, correspondente ao número de *threads* de *hardware* do sistema (48) sem qualquer política NUMA definida.

Como se pode observar, o desempenho dos modelos, tanto no TTFT como no TPOT, tende a melhorar com o aumento do número de *threads* (ou seja, o TTFT e o TPOT diminuem). No entanto, este ganho não é linear: por exemplo não se observam *speedups* de 4× no TPOT entre execuções com 12 e 48 *threads*, devido à contenção no barramento de memória, uma vez que a geração de *tokens* é um processo *memory-bound*.

Além disso, existem pequenas diferenças no desempenho dos modelos para números elevados de *threads*, que podem ser exploradas para otimizar os resultados. Recorrendo a uma análise estatística semelhante à aplicada aos compiladores, é possível normalizar cada medição em relação à melhor medição da sua categoria (modelo e *prompt*) e, subsequentemente, calcular a média geométrica destes valores normalizados para cada configuração de *threads* (Tabela 7).

<i>Threads</i>	NUMA	TTFT	TPOT
48	distributed	1.0424	1.0063
44	distributed	1.0309	1.0293
48	Não definida	1.1764	1.0502
36	distributed	1.1335	1.1158
24	distributed	1.4689	1.4236
12	distributed	2.4469	2.3407

Tabela 7: Média geométrica do TTFT e do TPOT, normalizados em relação à melhor configuração de *threading* por *benchmark*.

O objetivo é minimizar o TPOT, uma vez que o tempo de geração de *tokens* é, geralmente, o principal

contribuinte para o tempo de resposta dos modelos. Observa-se que o TPOT atinge o seu valor mínimo com 48 *threads* e a política de NUMA `distribute`, resultando numa redução de 4.4% no tempo médio por *token* gerado, em comparação com a mesma configuração de *threads* sem a política NUMA definida. Também se registaram ganhos, embora menores, na configuração de 44 *threads*.

Relativamente ao consumo de memória, como esperado, este mantém-se constante e independente do número de *threads* utilizados.

4.5 Sumário

Em suma, neste processo de otimização, foi possível obter ganhos de 68.5% (TPOT) em relação à pior configuração de compilação do `llama.cpp`, através da seleção do compilador Clang e da biblioteca de álgebra linear BLIS. Além disso, com ajustes à política de alocação NUMA, conseguiu-se ainda uma melhoria adicional de 4.4% (TPOT). No entanto, não foi possível obter ganhos de desempenho através da quantização dos modelos ou da alteração da sua forma de carregamento para memória.

5 Modelação de Desempenho

Para complementar a análise experimental, desenvolveu-se um modelo analítico simples, com o objetivo de prever o desempenho da inferência no nó ARM utilizado, com base em parâmetros do *hardware* e da carga de trabalho. O modelo proposto foi inspirado no utilizado pela ferramenta `canirun.ai` [25], cuja iniciativa despertou interesse aos autores enquanto entusiastas de execução local de LLMs. O foco foi a modelação do desempenho apenas da geração de *tokens*, uma vez que esta é geralmente responsável pela maior parte do tempo de resposta de uma LLM.

O modelo do `canirun.ai` baseia-se num “fator de eficiência” para estimar a largura de banda prática a partir do seu valor teórico. A principal diferença entre o modelo aqui apresentado e o original reside na omissão deste fator, e na utilização do *benchmark* STREAM [26] para medir a largura de prática da memória.

5.1 Formulação do Modelo

A inferência de LLMs em fase de *decoding* é uma operação *memory-bound* [5]: para gerar cada *token*, o processador precisa de percorrer a totalidade dos pesos do modelo, resultando numa intensidade aritmética reduzida. Esta característica justifica a adoção de um modelo baseado em largura de banda, que, embora mais simples, não considera diversos outros fatores do *hardware*, como processamento aritmético, caches, entre outros.

O modelo de desempenho proposto baseia-se no tamanho da LLM e na largura de banda da memória. O tempo mínimo para a geração de um *token* pode ser estimado como o tempo necessário para ler,

uma única vez, todos os pesos do modelo a partir da memória:

$$\text{TPOT} \geq \frac{M_{\text{RAM}}}{\text{BW}_{\text{mem}}}, \quad (1)$$

onde M_{RAM} representa o espaço de memória ocupado pelos pesos do modelo (em GiB), e BW_{mem} a largura de banda efetiva da memória do nó (em GiB/s), medida diretamente através do *benchmark* STREAM.

A dimensão em memória dos pesos do modelo é, por sua vez, estimada a partir do número de pesos e do número de *bits* por peso, por sua vez determinado pela quantização utilizada:

$$M_{\text{RAM}} = \frac{P \times b}{8 \times 1024^3} \cdot \alpha + c, \quad (2)$$

onde P é o número de parâmetros do modelo e b o número de *bits* por parâmetro. O elemento $\alpha = 1,1$ é um fator multiplicativo e o elemento $c = 0.5$ GiB é um termo aditivo: ambos contabilizam acessos a estruturas auxiliares também em memória, como a KV cache e *runtime buffers*. Estes valores correspondem aos utilizados em `canirun.ai`.

5.2 Validação Experimental

Para avaliar a precisão do modelo, as suas previsões foram comparadas com os valores de TPOT observados nos testes de desempenho da configuração Clang + BLIS, com 48 *threads* e política NUMA `distribute`. A largura de banda da memória do nó ARM, medida por meio do *benchmark* STREAM, foi de $\text{BW}_{\text{mem}} \approx 818$ GB/s. Os resultados são apresentados na Tabela 8.

Modelo	Quant.	Mem. (GiB)	Pred. (ms)	Obs. (ms)	Rácio	Erro (%)
Llama 3.1	Q3_K_M	2.794	4.534	60.900	13.43	92.55
Llama 3.1	Q4_K_M	3.725	5.879	75.900	12.91	92.25
Llama 3.1	Q8_0	7.451	11.258	74.600	6.63	84.91
Gemma 3	Q3_K_M	1.397	2.517	46.900	18.63	94.63
Gemma 3	Q4_K_M	1.863	3.189	52.500	16.46	93.92
Gemma 3	Q8_0	3.725	5.879	51.900	8.83	88.67
SmolLM 2	Q3_K_M	0.047	0.568	8.100	14.26	92.99
SmolLM 2	Q4_K_M	0.063	0.591	9.300	15.74	93.65
SmolLM 2	Q8_0	0.126	0.682	9.000	13.21	92.43
MSE						2090.371 ms ²

Tabela 8: Predições do modelo de desempenho analítico sem qualquer ajuste ($k = 1.0$).

Como se observa, os valores previstos subestimam os medidos, com r cios entre $7\times$ e $19\times$. Embora numericamente desfavor vel, este resultado   consistente com a natureza do modelo: a equa  o 1 fornece um limite inferior te rico, e n o uma estimativa do desempenho real. Para melhor caracterizar o desvio, ajustou-se um fator multiplicativo k , que minimiza o erro quadr tico m dio entre previs es e observa  es. O valor obtido de k foi $k = 9.27$. Em outras palavras, para aproximar o modelo da realidade, seria necess rio considerar uma perda de desempenho de $9.27\times$. Os resultados corrigidos s o apresentados na Tabela 9.

Modelo	Quant.	Mem. (GiB)	Pred. (ms)	Obs. (ms)	R�cio	Erro (%)
Llama 3.1	Q3_K_M	2.794	37.884	60.900	13.43	37.79
Llama 3.1	Q4_K_M	3.725	50.345	75.900	12.91	33.67
Llama 3.1	Q8_0	7.451	100.190	74.600	6.63	34.30
Gemma 3	Q3_K_M	1.397	19.192	46.900	18.63	59.08
Gemma 3	Q4_K_M	1.863	25.422	52.500	16.46	51.58
Gemma 3	Q8_0	3.725	50.345	51.900	8.83	3.00
SmolLM 2	Q3_K_M	0.047	1.131	8.100	14.26	86.04
SmolLM 2	Q4_K_M	0.063	1.341	9.300	15.74	85.58
SmolLM 2	Q8_0	0.126	2.182	9.000	13.21	75.75
MSE						388.822 ms²

Tabela 9: Predi  es do modelo de desempenho anal tico com multiplicador $k = 9.27$ ajustado por minimiza  o do MSE.

Mesmo ap s o ajuste, o modelo permanece pouco preciso, com erros que variam entre $\sim 3\%$ e $\sim 86\%$. Para investigar a origem destes desvios, o mesmo modelo foi aplicado a medi  es recolhidas localmente, num processador AMD Ryzen 7 5800H. Nesse cen rio, as raz es entre previs o e observa  o variaram entre $2\times$ e $3\times$, valores significativamente mais pr ximos do limite te rico do que aqueles obtidos no A64FX. Esta discrep ncia sugere que as limita  es do modelo n o residem apenas na sua formula  o, que n o contabiliza corretamente o custo de opera  es como dequantiza  o e acesso   KV Cache, mas tamb m na sua incapacidade de capturar caracter sticas espec ficas da arquitetura A64FX, como o acesso n o-uniforme   mem ria. Al m disso, particularidades da arquitetura ARMv8 podem tamb m estar envolvidas, uma vez que diferentes compiladores produziram desempenhos de gera  o de *tokens* radicalmente distintos, o que n o   esperado em *kernels memory-bound*.

Dada a magnitude dos desvios observados, n o   vi vel utilizar este modelo para identificar algum *bottleneck* dominante no processo de infer ncia ou orientar decis es de otimiza  o, como era objetivo do enunciado. Uma an lise rigorosa neste sentido requiriria um *profiling* detalhado dos *kernels* do `llama.cpp`.

6 Conclusões

Neste trabalho, foi possível otimizar a inferência de LLMs em ambientes de CPU, utilizando a *engine* de inferência `llama.cpp` em processadores Fujitsu A64FX. Foram exploradas diversas dimensões de otimização, destacando-se a configuração de compilação Clang + BLIS, que se revelou a mais promissora, com reduções no TPOT até 68.5%. O ajuste da política NUMA também resultou num ganho de desempenho de 4.4% (relativo à geração de *tokens*) em comparação com a configuração padrão do `llama.cpp`.

No entanto, as principais otimizações prenderam-se com a área de sistemas, e não nos parâmetros das LLMs em si. Por exemplo, não foi identificado qualquer padrão consistente no desempenho de modelos quantizados. Além disso, foram detetados problemas de replicabilidade e dimensão na metodologia de avaliação da qualidade das respostas da LLM, não tendo sido possível quantificar as diferenças na qualidade das respostas de modelos quantizados.

Para trabalho futuro, mais parâmetros das LLMs poderiam ser ajustados para avaliar a sua influência no desempenho: tamanho do *batch*, do μ -*batch*, e do contexto, bem como os tipos de dados da KV cache. Estes dois últimos parâmetros podem ainda ser úteis a reduzir o consumo de memória da KV cache, mitigando alguma instabilidade de *crashes* encontrada para modelos maiores. Nas dimensões da análise realizada, também seria pertinente avaliar o impacto de pedidos simultâneos no *throughput* total do sistema.

Do lado do `llama.cpp`, consideramos que as principais melhorias a fazer não se prendem com técnicas de inferência, mas sim com uma melhor configuração-padrão. Por exemplo, o uso de `--no-mmap` para modelos pequenos poderá reduzir a frequência de *crashes* encontrados, e o número máximo fixo de *tokens* poderá evitar problemas com temperatura e respostas infindas. No que diz respeito a técnicas de inferência, as principais limitações do `llama.cpp` estão relacionadas com inferência distribuída, uma funcionalidade cujo suporte é ainda muito recente. Funcionalidades como *prefill* e *decode* desagregados seriam bem vindas, especialmente em sistemas heterogêneos como o Deucalion [6, 27]. Em ambientes de nó único, *Multi-Token Prediction* seria uma otimização viável, com implementações iniciais a mostrar resultados promissores [28].

Quanto à modelação de desempenho, o modelo baseado em largura de banda memória da ferramenta `canirun.ai` foi adaptado, tendo o fator de eficiência sido substituído pela medição empírica da largura de banda de memória. No entanto, os resultados não foram satisfatórios: os valores previstos divergiram dos observados por fatores de $7\times$ a $19\times$, o que impossibilitou o uso do modelo para identificar *bottlenecks* dominantes ou orientar futuras otimizações.

Em suma, foi possível otimizar o desempenho de inferência de LLMs em processadores Fujitsu A64FX, mas acreditamos que ainda existe um grande potencial de otimização no ajuste de diversos parâmetros das LLMs, bem como muitas oportunidades para melhorar a modelação do desempenho de inferência.

Financiamento

Este trabalho utilizou horas de computação atribuídas pela bolsa 2025.00010.HPCVLAB.UMINHO, financiada pela Fundação para a Ciência e a Tecnologia (FCT).

Referências

- [1] K. Bhuyan, D. Marton, C. Guerreiro, and K. Meštrović, “The European High Performance Computing Joint Undertaking Infrastructure and Access Modes State-of-Play,” *Procedia Computer Science*, vol. 267, pp. 256-270, 2025.
- [2] National Advanced Computing Network, “DEUCALION.” [rnca.fccn.pt](https://rnca.fccn.pt/en/deucalion). Accessed: May 12, 2026. [Online.] Available: <https://rnca.fccn.pt/en/deucalion>
- [3] Fujitsu. *A64FX Microarchitecture Manual, 1.8.1.* (2022). Accessed: May 13, 2025. [Online]. Available: <https://github.com/fujitsu/A64FX>
- [4] The GGML authors. “llama.cpp.” [github.com](https://github.com/ggml-org/llama.cpp). Accessed: May 13, 2026. [Online.] Available: <https://github.com/ggml-org/llama.cpp>
- [5] P. Recasens et al., “Mind the Memory Gap: Unveiling GPU Bottlenecks in Large-Batch LLM Inference,” 2025, arXiv:2503.08311.
- [6] Perplexity. “Disaggregated Prefill and Decode.” Accessed: May 14, 2026. [Online.] Available: <https://www.perplexity.ai/hub/blog/disaggregated-prefill-and-decode>
- [7] A. Vaswani et al., “Attention Is All You Need,” 2017, arXiv:1706.03762.
- [8] F. Gloeckle, B. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, “Better & Faster Large Language Models via Multi-token Prediction,” 2024, arXiv:2404.19737.
- [9] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” 2023, arXiv:2309.06180.
- [10] G. Yu, J. S. Jeong, G. Kim, S. Kim, and B. Chun, “Orca: A Distributed Serving System for Transformer-Based Generative Models,” in *16th USENIX Symposium on Operating Systems Design and Implementation*, 2022, pp. 512-538.
- [11] Bartowski. “SmolLM-2-135M-Instruct-GGUF.” [huggingface.co](https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF). Accessed: May 13, 2026. [Online.] Available: <https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF>
- [12] LM-Kit. “Gemma-3-4B-Instruct-GGUF.” [huggingface.co](https://huggingface.co/lm-kit/gemma-3-4b-instruct-gguf). Accessed: May 13, 2026. [Online.] Available: <https://huggingface.co/lm-kit/gemma-3-4b-instruct-gguf>
- [13] Bartowski. “Meta-Llama-3.1-8B-Instruct-GGUF.” [huggingface.co](https://huggingface.co/bartowski/Meta-Llama-3.1-8B-Instruct-GGUF). Accessed: May 13, 2026. [Online.] Available: <https://huggingface.co/bartowski/Meta-Llama-3.1-8B-Instruct-GGUF>
- [14] L. Ouyang et al., “Training language models to follow instructions with human feedback,” 2022, arXiv:2203.02155.
- [15] The GGML authors. “LLAMA.cpp HTTP Server.” [github.com](https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md#post-v1completions-openai-compatible-completions-api). Accessed: May 13, 2026. [Online.] Available: <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md#post-v1completions-openai-compatible-completions-api>

- [16] “free(3) - Linux man page.” die.net. Accessed: May 13, 2026. [Online.] Available: <https://linux.die.net/man/3/free>
- [17] Free Software Foundation. “GCC, the GNU Compiler Collection.” Accessed: May. 13, 2026. [Online.] Available: <https://gcc.gnu.org>.
- [18] LLVM Developer Group. “Clang: a C language family frontend for LLVM.” Accessed: May. 13, 2026. [Online.] Available: <https://clang.llvm.org>.
- [19] The GGML authors. “Build llama.cpp locally.” github.com. Accessed: May 13, 2026. [Online.] Available: <https://github.com/ggml-org/llama.cpp/blob/master/docs/build.md>
- [20] X. Zhang, M. Kroeker. “OpenBLAS – An optimized BLAS library.” openmathlib.org. Accessed: May 13, 2026. [Online.] Available: <https://www.openmathlib.org/OpenBLAS>
- [21] F. van Zee and R. van de Geijn, “BLIS: A Framework for Rapidly Instantiating BLAS Functionality”, *ACM Transactions on Mathematical Software*, vol. 41, no. 3, pp. 1-14, 2015.
- [22] Fujitsu. “Fujitsu Software Technical Computing Suite V4.0L20 – Development Studio – BLAS LAPACK ScaLAPACK User’s Guide.” r-ccs.riken.jp. Accessed: May 13, 2026. [Online.] Available: <https://www.r-ccs.riken.jp/fugaku/docs/manual/en/lang/j2ul-2561-01enz0.pdf>
- [23] Fujitsu. “Fujitsu Software Technical Computing Suite V4.0L20 – Development Studio – C++ User’s Guide.” r-ccs.riken.jp. Accessed: May 13, 2026. [Online.] Available: <https://www.r-ccs.riken.jp/fugaku/docs/manual/en/lang/j2ul-2561-01enz0.pdf>
- [24] Kitware. “FindBLAS.” cmake.org. Accessed: May 13, 2026. [Online.] Available: <https://cmake.org/cmake/help/latest/module/FindBLAS.html>
- [25] M. Durán, “Can I Run AI locally?,” Accessed: May 14, 2026. [Online.] Available: <https://www.canirun.ai/why>
- [26] J. McCalpin, “Memory Bandwidth and Machine Balance in Current High Performance Computers,” in *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*, 1995.
- [27] G. Gerganov. “Server: disaggregated prefill/decode support.” github.com. Accessed: May 14, 2026. [Online.] Available: <https://github.com/ggml-org/llama.cpp/issues/21266>
- [28] Atomic Bot. “Multi-Token Prediction (MTP) for LLaMA.cpp - Gemma 4 speedup by 40%.” reddit.com. Accessed: May 14, 2026. [Online.] Available: https://www.reddit.com/r/LocalLLaMA/comments/1t6se6r/multitoken_prediction_mtp_for_llamacpp_gemma_4

A *Prompts* Utilizadas para Testes

A.1 *Prompts* de Pequena Dimensão

S0 What is the capital of France?

S1 Which country is the largest producer of cork?

S2 What is the chemical symbol of gold?

S3 Who painted "The Scream"?

S4 What is the second planet counting from the Sun?

A.2 *Prompts* de Média Dimensão

M0 Explain the key differences between machine learning and deep learning, and provide an example of each.

M1 Explain the difference between speed and velocity, and give an example to illustrate each concept.

M2 Explain the concept of supply and demand, and how it affects the price of goods.

M3 Explain the difference between matrix-matrix multiplication and matrix-vector multiplication. Give a practical application for each one.

M4 Explain the difference in chemical composition between acid and alkaline solutions. Give examples of both.

A.3 *Prompts* de Grande Dimensão

L0 The transformer architecture, introduced in the paper "Attention is All You Need" by Vaswani et al. in 2017, has become the foundation for most modern large language models. Unlike recurrent neural

networks, which process sequences sequentially, transformers process all tokens in parallel using self-attention mechanisms. This allows better parallelisation during training and has proven more effective at capturing long-range dependencies in text. When deploying transformers for inference on CPU, the main bottlenecks are the model's large memory footprint and the repeated loading of weights during the decoding phase. Based on this context, what are the main challenges when running large transformer models on CPU-only systems, and how might quantisation help address these challenges?

L1 The Paris Agreement aims to limit global warming to well below 2°C above pre-industrial levels, with efforts to cap it at 1.5°C. Achieving this requires rapid decarbonization, particularly in the energy sector. While solar and wind power have seen dramatic cost reductions, their intermittency remains a major challenge for grid stability. Energy storage solutions, such as lithium-ion batteries, are improving but still face limitations in scalability, cost, and environmental impact. Based on this context, what are the main obstacles to achieving a fully renewable energy grid, and how might advances in green hydrogen or next-generation battery technologies help address these challenges?

L2 Dark matter, which constitutes approximately 27% of the universe's mass-energy content, does not emit, absorb, or reflect light, making it invisible to traditional telescopes. Its existence is inferred from gravitational effects on visible matter, such as the rotation curves of galaxies and the behavior of galaxy clusters. Despite decades of research, the particle nature of dark matter remains unknown, with leading candidates including weakly interacting massive particles (WIMPs) and axions. Direct detection experiments, such as those using underground detectors or space-based observatories, have yet to yield conclusive evidence. Based on this context, what are the main challenges in detecting and identifying dark matter, and how might next-generation experiments or alternative theoretical models help resolve these challenges?

L3 Antimicrobial resistance (AMR) is a growing global health crisis, with bacteria, viruses, and fungi evolving to resist existing drugs. Overuse and misuse of antibiotics in healthcare and agriculture have accelerated this process, leading to infections that are increasingly difficult to treat. Low- and middle-income countries are particularly vulnerable due to limited access to diagnostics and alternative treatments. In this context, what are the main challenges in combating antimicrobial resistance?

L4 Portugal's higher education system is bifurcated into universities, which focus on theoretical and research-oriented education, and polytechnics, which emphasize applied, vocational training. While polytechnics play a crucial role in addressing labor market needs, they often face challenges such as lower prestige, limited research funding, and fewer opportunities for academic progression. With the growing demand for skilled technicians and the push for lifelong learning, how can polytechnics in Portugal enhance their status and better align their offerings with evolving industry requirements?

B Resultados dos Testes de Qualidade de Resposta

Prompt	Humberto	José Lopes	José Soares	Total
S0	2	2	2	6
S1	0	0	0	0
S2	2	2	2	6
S3	0	0	0	0
S4	0	0	0	0
M0	1	0	1	2
M1	0	0	0	0
M2	0	0	0	0
M3	0	0	0	0
M4	0	0	0	0
L0	0	0	1	1
L1	0	0	1	1
L2	0	0	0	0
L3	1	1	1	3
L4	0	0	0	0
Total	6	5	8	19

Tabela 10: Avaliação de qualidade do modelo SmolLM2-135M-Instruct-Q3_K_M.

Prompt	Humberto	José Lopes	José Soares	Total
S0	1	1	1	3
S1	0	0	0	0
S2	1	1	1	3
S3	0	0	0	0
S4	0	0	0	0
M0	0	0	1	1
M1	0	0	0	0
M2	0	0	0	0
M3	0	0	0	0
M4	1	1	0	2
L0	0	0	1	1
L1	2	1	1	4
L2	0	0	0	0
L3	0	0	1	1
L4	0	0	0	0
Total	5	4	6	15

Tabela 11: Avaliação de qualidade do modelo SmolLM2-135M-Instruct-Q4_K_M.

Prompt	Humberto	José Lopes	José Soares	Total
S0	2	2	2	6
S1	0	0	0	0
S2	2	2	2	6
S3	0	0	0	0
S4	0	0	0	0
M0	1	0	1	2
M1	1	0	0	1
M2	0	0	0	0
M3	0	0	0	0
M4	0	0	1	1
L0	0	0	0	0
L1	0	0	1	1
L2	0	0	0	0
L3	0	1	0	1
L4	0	0	1	1
Total	6	5	8	19

Tabela 12: Avaliação de qualidade do modelo SmolLM2-135M-Instruct-Q8_0.

Prompt	Humberto	José Lopes	José Soares	Total
S0	2	2	2	6
S1	1	1	1	3
S2	2	2	2	6
S3	2	2	2	6
S4	2	2	2	6
M0	2	1	2	5
M1	2	2	2	6
M2	1	2	2	5
M3	1	1	1	3
M4	2	1	2	5
L0	1	1	2	4
L1	2	2	2	6
L2	2	2	2	6
L3	2	2	2	6
L4	1	1	2	4
Total	25	24	28	77

Tabela 13: Avaliação de qualidade do modelo gemma-3-it-4B-Q3_K.M.

Prompt	Humberto	José Lopes	José Soares	Total
S0	2	2	2	6
S1	1	1	1	3
S2	2	2	2	6
S3	2	2	2	6
S4	2	2	2	6
M0	2	1	2	5
M1	2	2	2	6
M2	1	2	2	5
M3	2	2	2	6
M4	2	2	2	6
L0	2	2	2	6
L1	2	2	2	6
L2	2	2	2	6
L3	2	2	2	6
L4	2	2	2	6
Total	28	28	29	85

Tabela 14: Avaliação de qualidade do modelo `gemma-3-it-4B-Q4_K.M.`

Prompt	Humberto	José Lopes	José Soares	Total
S0	2	2	2	6
S1	1	1	1	3
S2	2	2	2	6
S3	2	1	2	5
S4	2	2	2	6
M0	2	1	2	5
M1	2	2	2	6
M2	1	2	2	5
M3	1	1	2	4
M4	2	2	2	6
L0	1	1	2	4
L1	2	2	2	6
L2	2	1	2	5
L3	2	2	2	6
L4	2	2	2	6
Total	26	24	29	79

Tabela 15: Avaliação de qualidade do modelo `gemma-3-it-4B-Q8_0.`

Prompt	Humberto	José Lopes	José Soares	Total
S0	2	2	2	6
S1	2	2	2	6
S2	2	2	2	6
S3	2	2	2	6
S4	2	2	2	6
M0	1	2	2	5
M1	2	2	2	6
M2	1	1	1	3
M3	1	1	2	4
M4	2	1	2	5
L0	2	1	2	5
L1	2	2	2	6
L2	2	2	2	6
L3	2	2	2	6
L4	2	2	2	6
Total	27	26	29	82

Tabela 16: Avaliação de qualidade do modelo Meta-Llama-3.1-8B-Instruct-Q3_K_M.

Prompt	Humberto	José Lopes	José Soares	Total
S0	2	2	2	6
S1	2	1	2	5
S2	2	2	2	6
S3	2	2	2	6
S4	0	0	0	0
M0	2	2	2	6
M1	2	2	2	6
M2	1	1	1	3
M3	2	2	2	6
M4	2	2	2	6
L0	1	1	2	4
L1	2	2	2	6
L2	2	2	2	6
L3	1	1	2	4
L4	2	2	2	6
Total	25	24	27	76

Tabela 17: Avaliação de qualidade do modelo Meta-Llama-3.1-8B-Instruct-Q4_K_M.

Prompt	Humberto	José Lopes	José Soares	Total
S0	2	2	2	6
S1	2	2	2	6
S2	2	2	2	6
S3	1	2	2	5
S4	2	2	2	6
M0	2	2	2	6
M1	1	1	1	3
M2	2	2	2	6
M3	1	1	1	3
M4	2	2	1	5
L0	2	2	2	6
L1	2	2	2	6
L2	2	2	2	6
L3	1	2	2	5
L4	2	2	2	6
Total	26	28	27	81

Tabela 18: Avaliação de qualidade do modelo Meta-Llama-3.1-8B-Instruct-Q8_0.